Lecture 04: Structures, Enumerators, Exceptions & Basic IO

Objectives:

- Learn about structures (struct) and Enumerators (enum)
- Learn how to raise and handle exceptions
- Learn how to do basic file IO using Streams.

1. Structures

struct is similar to a class in its declaration and in terms of the members it can have.

Like a class, struct members can be *constructors*, *constants*, *fields*, *methods*, *properties*, *indexers*, *operators*, and *nested types*.

However, structs are **value types** not reference types – hence they are sometimes referred to as a lightweight version of a class. For this reason, structs don't incur the overhead associated with reference objects except when boxed.

On the other hand, structs have some severe limitations that restrict their usage to very specialized situations.

The following is a simple example of a struct.

```
structColor
{
publicintRed;
publicintGreen;
publicintBlue;
}
```

A variable of the above Struct can be declared and initialized as follows:

```
Colorrgb;
rgb.Red=0;
rgb.Green=0;
rgb.Blue=0;
```

Notice that we didn't have to instantiate the struct (via the new keyword). This is because, as a value type, a struct is allocated memory once it's declared.

A struct can also be initialized using new.

```
ColorrgbColor=newColor();
Console.WriteLine(rgbColor.Red);
```

The above uses the implicit default constructor, which initializes the fields of a struct to their default values.

Note that explicit default constructors are not allowed for structs. However, explicit non-default constructors are allowed as show below:

```
struct Color
{
    public int Red;
    public int Green;
    public int Blue;

    public Color(int red, int green, int blue)
    {
        Red = red;
        Green = green;
        Blue = blue;
    }

    public override String ToString()
    {
        return "(Red="+ Red + ", Green=" + Green + ", Blue=" +
Blue+")";
    }
}
```

Another limitation for structs is that they can neither be derived from, nor can they derive from other structs - except from the **System.Value** type, which all structs implicitly derived from.

However, a struct can implement an interface.

Although structs are more efficient than classes, their limitations restrict their usage. As a general rule, you should use structs only when:

- The data being contained is very small. Examples of this are structs that hold **Point** values (x and y), **RGB Color** values.

- The struct will contain few or even no methods to access or modify the contained data.

2. Enumerators

An enumeration (enum) is a special form of **value type**, which is used to assign symbolic names to a restricted set of values of an underlying integral type (*int, uint, byte, sbyte, short*, etc – except *char*).

An enumeration type has a **name**, an **underlying type**, and a **set of fields**. The fields are static literals, each of which represents a constant.

For example, the following code declares an enum representing week days.

```
public enum WeekDay {
    Sunday,
    Monday,
    Tuesday,
    Wednesday,
    Thursday,
    Friday,
    Saturday
}
```

This code is affected by two default settings:

Firstly, the default value type for enumerators is **int**.

Secondly, by default, the first literal specified is set to **0** and this value is then incremented for each subsequent literal.

Hence, given the above, **WeekDay.Sunday** equals the integer **0** and **WeekDay.Saturday** equals the integer **6**.

The following code illustrates how to force a different numeric type, and how to change the value assigned to the first literal:

```
public enum WeekDay : byte {
    Sunday = 1,
    Monday,
    Tuesday,
    Wednesday,
    Thursday,
    Friday,
    Saturday
}
```

The advantage of using enum is that they make a program more readable and less error prone than using the underlying values directly. For example, compare the following two methods:

```
public static bool IsWeekEnd(WeekDay day) {
    return day == WeekDay.Thursday || day == WeekDay.Friday;
}
```

```
public static bool IsWeekEnd(int day) {
    return day == 5 || day == 6;
}
```

Clearly, the first method (using enum) is more readable.

Also since the second method takes an *int* as argument, it is possible to call it with any *int* value – which could lead to errors -- whereas, the first method can only take one of its specified values.

The **System.Enum** class provides some methods that can be used to manipulate enum types. For example, the following prints the literals and the values of the **WeekDay** enumerator.

```
String[] names = Enum.GetNames(typeof(WeekDay));
foreach(string s in names)
    Console.WriteLine(s);

byte[] values = (byte[]) Enum.GetValues(typeof(WeekDay));
foreach(byte i in values)
    Console.WriteLine(i);
```

3. Exception Handling:

Like Java, C# handles errors using exceptions. However, in C#, all exceptions are run-time exceptions derived from the **System.Exception** class.

A method uses the ***throw*** keyword to raise an exception when an error condition occurs. For example, the following divide method raises a **DivideByZeroException** when the second number is zero.

```
using System;

public class  TestException {

     public static double Divide(double x, double y) {
          if (y==0)
               throw new DivideByZeroException("Can't divide by
zero");
          else
               return x/y;
     }

     public static void Main() {
          try {
               Console.Write("Enter first value: ");
               double x = double.Parse(Console.ReadLine());
               Console.Write("Enter second value: ");
               double y = double.Parse(Console.ReadLine());
               Console.WriteLine(x + "/" + y + " = "+ Divide(x, y));
          }
          catch (DivideByZeroException e) {
               Console.WriteLine(e.Message);
          }
          catch (FormatException) {
               Console.WriteLine("Format exception occurs");
          }
          catch {
               Console.WriteLine("Some Other Exception occurs");
          }
     }
}
```

Note that the same **try-catch-finally** (as in Java) is used to handle the exception. That is, the code that may result in exception is placed in the **try** block. The **catch** block is used to handle the exception if it occurs. The optional **finally** block is used to place a code that must be executed whether an exception is raised or not.

Note also that there are many options for the catch block:

If there is no need to refer to the exception instance, then there is no need to declare a variable to receive it.

In fact, the whole **catch** expression can be omitted if there is no need to refer to the resulting exception. This will catch all exceptions even those that are not derived from the **System.Exception** class.

Finally, we note that c# does not have the ***throws*** keyword. If a method does not wish to handle an exception, it just ignores it, and it will automatically be forwarded to a higher method.

4. Basic Stream IO:

In C#, like in Java, IO operations are designed around streams - a sequence of bytes traveling from a source to a destination.

In what follows, we look at the classes that are used to handle basic IO operations. Please note that Network programming is mainly about Protocols and IO, so it is important to understand these IO classes.

The Stream class

The InputSream and OuputSream classes that we know in Java are unified in C# into a single abstract class, **Stream**. So the Stream class is used to represent a sequence of bytes, both from a source and to a destination. The source/destination can be a file, a network connection, or some IO device.

The basic methods of the Stream class are shown below:

int Read(in byte[] *buffer*, int offset, int count)	Reads *count* bytes from a source and stores the bytes read into the *buffer* array, stating at index *offset*. It returns the number of the actual bytes read or 0

int ReadByte()	Reads one byte from the stream and moves the position by one byte, or returns -1 if at the end of the stream.
void Write(in byte[] *buffer*, int offset, int count)	Writes count bytes from the buffer array, stating at index offset, into a destination
void WriteByte()	Writes a byte to the current position in the stream and advances the position within the stream by one byte.
void Flush()	Clears all buffers for this stream and causes any buffered data to be written to the underlying device.
void Close()	Closes the current stream and releases any resources associated with the stream.

The FileStream class

FileStream is an example of a concrete class that extends the Stream class. As the name suggests, it allows streams of bytes to be transferred between a source file and a destination file. Another concrete class that extends the Stream class is the **NetworkStream** class. A lot of our network programs will use this class.

Unlike Java, which provides separate classes for file input and file output (FileInputStream and FileOutputStream), In C#, **FileStream** is used for both input and output.

The following are some of the constructors of the FileStream class:

```
public FileStream(string path, FileMode mode)
public FileStream(string path, FileMode mode, FileAccess access)
public FileStream(string path, FileMode mode, FileAccess access,
FileShare share)
```

The first constructor allows the file, **path**, to be opened for read/write access. FileMode is an enumeration type that is used to indicate how the operating system should open the file. Its

values are:

Append	Opens the file if it exists and seeks to the end of the file, or creates a new file. **FileMode.Append** can only be used in conjunction with **FileAccess.Write**. Any attempt to read fails and throws an **ArgumentException**.
Create	Specifies that the operating system should create a new file. If the file already exists, it will be overwritten.
CreateNew	Specifies that the operating system should create a new file. If the file already exists, an **IOException** is thrown.
Open	Specifies that the operating system should open an existing file. A **FileNotFoundException** is thrown if the file does not exist.
OpenOrCreate	Specifies that the operating system should open a file if it exists; otherwise, a new file should be created.
Truncate	Specifies that the operating system should open an existing file. Once opened, the file should be truncated so that its size is zero bytes. Attempts to read from a file opened with Truncate causes an exception.

The second constructor has an additional argument of type **FileAccess**. This is used to specify whether the file is being opened for reading, writing or both. The values of the **FileAccess** enumeration are: ***Read***, ***Write*** and ***ReadWrite***.

The third constructor has an additional argument of type **FileShare**. This is used to specify how other threads or processes should be allowed access to the same file. The values of the FileShare enumeration are: ***Inheritable***, ***Read***, ***Write***, ***ReadWrite***.

The methods of **FileStream** class are essentially the same as those of the **Stream** class.

Some useful properties of the **FileStream** class are: ***CanRead***,

CanWrite, ***Length*** (size) and ***Position***.

The following example shows how to use the FileStream class.

```
using System;
using System.IO;

public class StreamFileIO {
     public static void Main() {
          try {
               FileStream inFile = new FileStream("saudiflag.gif",
FileMode.Open);
               FileStream outFile = new FileStream("flagcopy.gif",
FileMode.Create);
               byte[] buffer = new byte[1024];

               while (inFile.Position < inFile.Length) {
                    int read = inFile.Read(buffer, 0,
buffer.Length);
                    outFile.Write(buffer, 0, read);
               }
               inFile.Close();
               outFile.Close();
          }
          catch (FileNotFoundException) {
               Console.WriteLine("Sorry, File not found");
          }
          catch (Exception e) {
               Console.WriteLine("Sorry, Exception: "+e);

          }

     }
}
```

Text IO

For the purpose of Text IO, C# has separate classes for input and output, namely, **StreamReader** and **SreamWriter**, respectively.

SreamReader:

The most common constructors for the StreamReader class are:

public StreamReader(string path) //using UTF-8 as the default encoding scheme.
public StreamReader(string path, Encoding encoding)

Some of the methods of StreamReader class are (in addition to those inherited from the Stream class):

int Read()	Reads a single character, returns –1 if end
int Peek()	Returns the next character without reading end of stream.
void Read(char[ ], int offset, int count)	Reads an array of characters
string ReadLine()	Reads a line of characters
string ReadToEnd()	Reads from the current position to end

StreamWriter:

The most common constructors for the StreamWriter class are:

public StreamWriter(string path) //using UTF-8 as the default encoding scheme.
public StreamWriter(string path, bool append) //for appending
public StreamWriter(string path, Encoding encoding)
public StreamWriter(string path, bool append, Encoding encoding) //for appending

The basic methods of the **StreamWriter** class (in addition to those inherited from the Stream class) are **Write** and **WriteLine**. These are overloaded to accept **char, char[], string**, and each of the **primitive types**.

The following example shows how to read and write to a text file.

```
using System;
using System.IO;

public class TextFileIO {
     public static void Main() {
          try {
               StreamReader inFile = new
StreamReader("SWE344.txt");
               StreamWriter outFile = new
StreamWriter("output.txt");

               String line = null;
               while ((line = inFile.ReadLine()) != null) {
                    Console.WriteLine(line);
                    outFile.WriteLine(line);
```

```
                }
                inFile.Close();
                outFile.Close();
            }
            catch (FileNotFoundException) {
                Console.WriteLine("Sorry, File not found");
            }
            catch (Exception e) {
                Console.WriteLine("Sorry, Exception: "+e);

            }
        }
}
```

File Handling:

C# provides a number of classes for handling Files at the file system level. Operations such as ***Create, Copy, Move, Delete*** for both files and directories are provided through the following classes:

File, FileInfo, Directrory and **DirectoryInfo**.

We explain the File class and allow you to check the others from the documentation.

The File class

All the methods in the File class are static, so there is no constructor. Some of the methods are:

static FileStream Create(String path)	Creates the file specified by path and returns its FileStream which can be used to write streams to the file.
static StreamWriter CreateText(string path)	Creates the file specified by path, and returns a StreamWriter which can be used to write text to the file.
static StreamWriter AppendText(string path)	Opens a file for appending text.
static FileStream Open(string path, FileMode mode)	opens a FileStream on the file specified by path using one of the FileStream open methods described earlier.
static FileStream Open(string path, FileMode mode, FileAccess access)	

static FileStream Open(string path, FileMode mode, FileAccess access, FileShare share)	
static StreamReader OpenText(string path)	Opens a file for text input.
static void Copy(string source, string destination)	Copies source file as destination
static void Move(string source, string destination)	Moves source file to destination
static void Delete(string path)	Deletes the file specified by path
static bool Exists(string path)	Checks if the file specified by path exists

Note: A problem with the **File** class is that each time one of its methods is called, the system must check that the user has permission on the file system before such operation is allowed. This can lead to inefficiency if there is frequent calls to the methods.

To solve this problem, C# provides the **FileInfo** class with similar set of methods, but which are non-static. In this case, permission is only checked at the point of creating an instance of FileInfo.

The **Directory** class provides static methods similar to those of **File** class, but for manipulating directories. The **DrectoryInfo** class is also provided to provide instance methods, which are more efficient.

The following example shows how to use the methods of the File class.

```
using System;
using System.IO;
public class CopyFile {
     public static void Main() {
          try {
               File.Copy("saudiflag.gif", "saudiflag2.gif");
          }
          catch (FileNotFoundException) {
               Console.WriteLine("Sorry, File not found");
          }
          catch (Exception e) {
               Console.WriteLine("Sorry, Exception: "+e);

          }
     }
```

```
}
```